

Implementation of an optimized pipelined AES-128 Encryption Module using Verilog for PYNQ Board

Raj Vardhan Singh (251110060)
Shrey Sharma (251110068)
Vikrant Nagpal (251110083)
Hemant Jagtap (251110037)
Abhishek Dixit (251110003)

November 9, 2025

1 Baseline Design

The baseline AES-128 design provided consists of a main Verilog module: `AESEncrypt128_DUT` and other submodules. The architecture of the design is explained in subsequent sections.

1.1 Baseline Architecture

The `AESEncrypt128_DUT` module serves as the top-level wrapper. It instantiates two sub-modules:

- **KeyExpansion (ke):** This module is responsible for generating all 11 round keys (for AES-128, $N_r = 10$) from the single 128-bit input key.
- **AESEncrypt (aes):** This module performs the actual encryption, using the round keys generated by `KeyExpansion`.

There are two major performance bottlenecks:

1. **Combinatorial Key Expansion:** The `KeyExpansion` module is purely combinatorial. It takes the 128-bit key as input and produces all 1408 bits ($11 \text{ rounds} \times 128 \text{ bits/round}$) of `allKeys` as a single, wide output wire. This results in a very long critical path which will limit the maximum clock frequency ($f_{\max}$) of the entire design.
2. **Multi-Cycle Encryption:** The `AESEncrypt` module is not pipelined. It is an iterative design that reuses a single set of round-function hardware (`SubBytes`, `ShiftRows`, `MixColumns`, `AddRoundKey`) over multiple clock cycles. The output is controlled by the `roundCount` register.

1.2 Baseline Operational Flow

The `AESEncrypt` module's operation is sequential and takes 11 cycles to process one block of data. The flow is as follows:

1. **Initial State (Reset):** `roundCount` is set to 1. The `done` signal is low.
2. **Cycle 1 (Round 0):** The condition (`roundCount == 1`) selects the raw input data as the input to the `AddRoundKey` module.

$$\text{stateOut} = \text{data} \oplus \text{RoundKey}[0]$$

On the clock edge, this result is captured in the `state` register, and `roundCount` increments to 2.

3. **Cycles 2–10 (Rounds 1–9):** The datapath is:

$$\text{state} \rightarrow \text{SubBytes} \rightarrow \text{ShiftRows} \rightarrow \text{MixColumns}$$

The `MixColumns` output (`mixColumnsWire`) is selected as the input to `AddRoundKey`.

$$\text{stateOut} = \text{Mix}(\text{Shift}(\text{Sub}(\text{state}))) \oplus \text{RoundKey}[\text{roundCount} - 1]$$

The result is captured in `state`, and `roundCount` increments.

4. **Cycle 11 (Final Round 10):** For the final round, `MixColumns` is skipped. The datapath is:

$$\text{state} \rightarrow \text{SubBytes} \rightarrow \text{ShiftRows}$$

The `ShiftRows` output (`shiftRowsWire`) is selected as the input to `AddRoundKey`.

$$\text{stateOut} = \text{Shift}(\text{Sub}(\text{state})) \oplus \text{RoundKey}[10]$$

The final ciphertext is captured in `state`, and `roundCount` increments to 12.

When the condition (`roundCount > Nr + 1`) becomes true, the `done` signal asserts high. The module now holds the final result in `state` but cannot accept new data until it is reset.

1.3 Limitations of the design

Although this design requires much less resource when implemented in the PYNQ board, it does not meet the performance requirements due to the following limitations

- **Low Throughput:** It takes 11 cycles to encrypt a single 128-bit block. A new encryption cannot begin until the first is complete. The resulting throughput is:

$$\text{Throughput} = \frac{128 \text{ bits}}{11 \text{ cycles}}$$

- **Long Critical Path:** The design’s maximum clock frequency ($f_{\max}$) is limited by two primary factors:

1. The massive combinatorial `KeyExpansion` module.
2. The datapath within `AESEncrypt`:

$$\text{reg} \rightarrow \text{SubBytes} \rightarrow \text{ShiftRows} \rightarrow \text{MixColumns} \rightarrow \text{MUX} \rightarrow \text{AddRoundKey} \rightarrow \text{reg_setup}$$

This entire path must complete within a single clock cycle.

2 Pipelined Design (Design 1)

To obtain high throughput, the baseline encryption module was completely redesigned into a fully pipelined architecture. The goal is to achieve a throughput of one 128-bit block per clock cycle after an initial latency.

2.1 Design Architecture: Pipelined Encryption Datapath

The new `AESEncrypt` module implements a deep pipeline, where each stage of the AES algorithm is separated by a set of registers. This allows a new data block to enter the pipeline on every clock cycle, moving one stage forward as the previous block advances.

The core of this design is the pipeline register array:

$$\text{reg } [127:0] \text{ state_pipe } [0:\text{Nr}];$$

This array defines 11 pipeline stages (for $N_r = 10$), holding the intermediate state of 11 different data blocks simultaneously.

2.1.1 Architecture Overview

The architecture is structured as follows:

- **Stage 0 (Initial Round):** Performs the initial `AddRoundKey` (pre-whitening).

$$\text{stage_out}[0] = \text{data_in} \oplus \text{RoundKey}[0]$$

This result is stored in `state_pipe[0]` on the next clock edge.

- **Stages 1–9 (Main Rounds 1–9):** These stages are instantiated using a `generate` loop. Each stage takes its input from the previous pipeline register (`state_pipe[r-1]`).

$$\text{stage_out}[r] = \text{AddRoundKey}(\text{MixColumns}(\text{ShiftRows}(\text{SubBytes}(\text{state_pipe}[r-1]))), \text{RoundKey}[r])$$

The result is stored in `state_pipe[r]` on the next clock edge.

- **Stage 10 (Final Round 10):** Implements the final round, which omits the `MixColumns` step. It takes its input from `state_pipe[9]`.

$$\text{stage_out}[10] = \text{AddRoundKey}(\text{ShiftRows}(\text{SubBytes}(\text{state_pipe}[9])), \text{RoundKey}[10])$$

The result is stored in `state_pipe[10]` on the next clock edge.

- **Output:** The final encrypted ciphertext is the output of the last pipeline register.

$$\text{assign data_out} = \text{state_pipe}[10];$$

2.2 Performance Impact

This pipelined architecture successfully decouples the rounds, dramatically increasing the throughput of the system.

- **Initial Latency:** It takes 11 clock cycles for the first data block to propagate through all 11 pipeline stages (from `data_in` to `data_out`).
- **Throughput:** After this initial latency, the pipeline is full. A new, fully encrypted 128-bit block emerges from `state_pipe[10]` on every subsequent clock cycle. This achieves the goal of single-cycle throughput.
- **Area Trade-off:** Implementing this design means creating separate hardware for each stage, hence increasing the resource utilization significantly.

2.3 Remaining Bottleneck: Combinatorial Key Expansion

Although the `AESencrypt` module is now highly optimized, in terms of single cycle output, the overall design's critical path is still dominated by the key expansion logic.

The `KeyExpansion` (`ke`) module remains the original combinatorial version. Consequently, while the encryption pipeline achieves high throughput, the top-level `AESencrypt128_DUT` design is still constrained by the following limitation:

- **New Critical Path:** The longest path in the entire design is the massive combinatorial path inside the `KeyExpansion` module, which must generate all 11 round keys (1408 bits) from the 128-bit input key in zero time (combinatorially).

This long path will force the system to run at low frequency, canceling the single cycle output generation advantage gained from pipelining the data path.

3 Parallel Key Expansion (Design 2)

Instead of pre-calculating the keys, the key expansion logic is integrated directly into the main encryption pipeline. The round key for each stage is generated in parallel with the data transformation of that same stage.

This is achieved by:

- Removing the monolithic `KeyExpansion` module.
- Adding a new pipeline register array, `key_pipe [0:Nr]`, to hold the intermediate keys. This *key pipeline* runs in parallel with the *data pipeline* (`state_pipe`).
- Instantiating the `KeyExpansionRound` logic directly inside each pipeline stage (from Stage 1 to Stage 10).

3.1 Architectural Implementation

The flow of data and keys is now perfectly synchronized:

- **At Clock Edge (Cycle 0):**
 - `state_pipe[0]` registers the result of $\text{data_in} \oplus \text{key}$.
 - `key_pipe[0]` registers the initial key.
- **In Stage 1 (Combinatorial Logic):**
 - **Data Path:** The main AES round logic (`SubBytes`, `ShiftRows`, `MixColumns`) processes the data from `state_pipe[0]`.
 - **Key Path:** A `KeyExpansionRound` instance uses the key from `key_pipe[0]` to generate `key_out[1]` (the Round 1 key).
 - **Final Step:** The `AddRoundKey` module combines the results from both parallel paths.
- **At Clock Edge (Cycle 1):**
 - `state_pipe[1]` registers the final output of Stage 1.
 - `key_pipe[1]` registers the newly generated `key_out[1]`.

This process repeats for all 11 stages. The logic for any given stage r (where $r > 0$) is summarized as:

$$\text{key_pipe}[r] = \text{KeyExpansionRound}(\text{key_pipe}[r - 1]), \quad \text{state_pipe}[r] = \text{AESRound}(\text{state_pipe}[r - 1], \text{key_pipe}[r])$$

Thus, the key for stage r is generated from the previous stage's key (`key_pipe[r-1]`) at the same time as the data is being processed.

3.2 Performance and Resource Analysis

This architecture is the key to achieving the design's high performance:

- **Critical Path:** The 10-round key expansion path is eliminated. The overall design's critical path is now the longer of the two parallel paths within a single pipeline stage:

Data Path: `reg` $\rightarrow$ `SubBytes` $\rightarrow$ `ShiftRows` $\rightarrow$ `MixColumns` $\rightarrow$ `AddRoundKey` $\rightarrow$ `reg_setup`

Key Path: `reg` $\rightarrow$ `KeyExpansionRound` $\rightarrow$ `AddRoundKey` $\rightarrow$ `reg_setup`

This is drastically shorter than the original design and allows for a much higher maximum clock frequency ($f_{\max}$).

- **Initial Latency:** Because the keys are generated exactly when they are needed, no additional latency is incurred. The initial latency remains 11 cycles—the depth of the data pipeline.

- **Throughput:** After the 11-cycle latency, the throughput remains one 128-bit block per clock cycle.
- **Area Trade-off:** This performance is achieved through a classic speed-for-area trade-off. Because of additional register usage to hold the key for each round, the overall area is increased slightly.

4 Fine-Grained Pipelining (Design 3)

Previous architecture successfully implemented parallel data and key pipelines, achieving one 128-bit block per cycle throughput. However, timing analysis revealed that the design was still constrained by the longest combinational path within a single pipeline stage.

The limiting path was the maximum of the following two:

Data Path: $\text{reg} \rightarrow \text{SubBytes} \rightarrow \text{ShiftRows} \rightarrow \text{MixColumns} \rightarrow \text{AddRoundKey} \rightarrow \text{reg_setup}$

Key Path: $\text{reg} \rightarrow \text{KeyExpansionRound} \rightarrow \text{AddRoundKey} \rightarrow \text{reg_setup}$

Implementation results showed that the logic within the **SubBytes** module and the **KeyExpansionRound** module contributed the largest delays. Total critical path of the AES round was higher than that of Key expansion because it involves **SubBytes** as well as other operations. To increase the maximum clock frequency ($f_{\max}$), the AES round was pipelined to perform the complete round operation in two cycles. This reduced the maximum critical path delay. And to maintain the alignment of AES round Key expansion, additional registers were also used in the **KeyExpansionRound** module, so that correct key goes into **AddRoundKey** module.

4.1 Pipelining Sub-Modules

- **KeyExpansionRound:** A register (**keyOut**) was added at the module's output, converting it into a single-cycle pipeline stage.

Cycle 1 (Combinational): $\text{keyIn} \rightarrow (\text{RotWord}, \text{SubWord}, \text{XORs}) \rightarrow \text{keyOut_comb}$

Cycle 1 (Register): $\text{keyOut} \leftarrow \text{keyOut_comb}$

- **SubBytes:** Similarly, a register (**subBytes**) was added at the output to form a single-cycle pipeline stage for the 16-byte substitution logic.

Cycle 1 (Combinational): $\text{oriBytes} \rightarrow (16 \times \text{S-Box Lookups}) \rightarrow \text{subBytes_comb}$

Cycle 1 (Register): $\text{subBytes} \leftarrow \text{subBytes_comb}$

4.2 New Pipeline Architecture: Two-Stage Rounds

This modification fundamentally changed the pipeline structure. Each of the ten main AES rounds (Rounds 1–10) was split into two sub-stages, effectively doubling the pipeline depth.

For any given round r ($r > 0$), the operation sequence is as follows:

Stage r -A (Sub-Stage 1):

- The **SubBytes** and **KeyExpansionRound** modules operate in parallel.
- **SubBytes** computes substitutions on the data from **state_pipe[r-1]**.
- **KeyExpansionRound** generates the next round key from **key_pipe[r-1]**.
- At the clock edge, their results are stored in internal registers.

Stage r -B (Sub-Stage 2):

- The combinational `ShiftRows`, `MixColumns`, and `AddRoundKey` modules execute.
- `ShiftRows` and `MixColumns` process the registered output from `SubBytes`.
- `AddRoundKey` combines this result with the registered key from `KeyExpansionRound`.
- At the clock edge, the results are stored in `state_pipe[r]` and `key_pipe[r]`.

4.3 Performance Impact Analysis

This fine-grained pipelining introduces a classic trade-off between latency, area, and frequency.

- **Critical Path:** The original long critical path is successfully divided. The new critical path becomes the longest of the following much shorter paths:

`reg` $\rightarrow$ `SubBytes_logic` $\rightarrow$ `reg_setup`
`reg` $\rightarrow$ `KeyExpansionRound_logic` $\rightarrow$ `reg_setup`
`reg` $\rightarrow$ `ShiftRows` $\rightarrow$ `MixColumns` $\rightarrow$ `AddRoundKey` $\rightarrow$ `reg_setup`

This results in a significantly higher achievable clock frequency ($f_{\max}$).

- **Throughput:** Although the pipeline depth has increased, the datapath remains fully feed-forward. Once filled, the pipeline still outputs one 128-bit block per clock cycle:

$$\text{Throughput (Gbps)} = 128 \text{ bits} \times f_{\max}$$

Thus, the higher clock frequency directly increases throughput.

- **Area:** The trade-off for improved performance is an increase in area. Which comes because of increased usage of registers to hold the intermediate outputs in `KeyExpansionRound` and `SubBytes` modules.
- **Initial Latency:** The total latency for encrypting the first block increases as follows:

Stage 0 (Initial `AddRoundKey`): 1 cycle
Stages 1–10 (Main Rounds): 10 rounds $\times$ 2 cycles/round = 20 cycles
Total Initial Latency: 1 + 20 = 21 cycles

This increase from 11 to 21 cycles represents the necessary trade-off to achieve the highest possible clock speed and throughput.

5 Deep Pipelining (Design 4)

The fully optimized, fine-grained pipeline design was integrated into a PYNQ board system using `Vitis`. The complete system, including the AES encryption IP and the necessary AXI interconnect IPs, was implemented in `Vivado` to determine the final, achievable performance metrics.

5.1 System-Level Bottleneck

The design's maximum frequency ($f_{\max}$) was no longer limited by the AES design itself. The fine-grained pipelining successfully shortened the critical path of the encryption logic to operate well above 250 MHz.

However, when placed and routed within the PYNQ Zynq-7000 system, the new critical path was found in the AXI interconnect IP and other board-level infrastructure required to connect the core to the processing system. As a result, the target 250 MHz clock frequency could not be met.

The highest achievable clock frequency at which the entire system successfully met timing was **215 MHz**.

In order find out the theoretical maximum frequency of the design, a synthesis of just the AES core (without the PYNQ system overhead) was performed. This test revealed that the maximum frequency ($f_{\max}$) of the design was **275 MHz**. The critical path remained the longest single-cycle path, which was the logic within the `KeyExpansionRound` modules.

One final optimization was implemented: **intra-module pipelining**. This technique involves adding registers inside the `SubBytes` and `KeyExpansionRound` modules to transform them into multi-cycle components.

5.2 Multi-Cycle Sub-Module Design

KeyExpansionRound (2-cycle): The module's logic was split into two internal stages:

- **Stage 1:** Performs the computationally expensive S-Box lookup (`SubWord`) and Rcon logic. The results are latched into new internal registers: `w3Sub_reg`, `words_reg`, and `roundConstant_reg`.
- **Stage 2:** Performs the simple, high-speed XOR operations using the values from the internal registers. The result is latched at the module's output (`keyOut`).

SubBytes (2-cycle): To maintain synchronization between the data and key pipelines, the `SubBytes` module was also converted into a 2-cycle component:

- **Stage 1:** Performs the 16 parallel S-Box lookups. The results are latched into an internal register (`subBytes_stage1`).
- **Stage 2:** The data from `subBytes_stage1` is passed to the final output register (`subBytes`).

5.3 New Pipeline Architecture and Latency

This modification effectively turns each of the 10 main AES rounds into a **3-cycle operation**, summarized as follows:

This change significantly impacts the initial latency:

$$\text{Stage 0 (Initial AddRoundKey)} = 1 \text{ cycle}$$

$$\text{Stages 1–10 (Main Rounds)} = 10 \text{ rounds} \times 3 \text{ cycles/round} = 30 \text{ cycles}$$

$$\text{New Initial Latency: } 1 + 30 = 31 \text{ clock cycles}$$

This represents an increase from the original 21-cycle latency. Synthesis of this design confirmed that the critical path was significantly reduced, allowing the design to achieve a new maximum frequency of:

$$f_{\max} = 375 \text{ MHz}$$

However, due to increase in the number of register usage, the area also increases slightly.

6 Results and Discussions

The metrics used to evaluate performance of the design are:

$$\text{Latency} = \frac{\text{Number of cycles to get first output}}{F_{\max}}$$

$$\text{Throughput} = \frac{128\text{bits} * F_{\max}}{\text{Number of cycles to get subsequent outputs}}$$

$$\text{Area} = 128\text{Number of LUTs} + \text{Number of Registers}$$

The performance of each design is summarised in table 1.

Table 1: AES Design Comparison Metrics

Design	Latency (ns)	Fmax (MHz)	Throughput (Gbps)	LUTs (%)	Registers (%)
Baseline	440	10	0.116	7.68	0.88
Design 1	440	25	3.2	18.8	2.07
Design 2	58.67	187.5	24	19.34	3.28
Design 3	76.36	275	35.2	19.16	5.68
Design 3	82.67	375	48	17.88	8.39

Final System Performance

Implementation of the designs showed that both the optimized designs (3 and 4) were limited in the frequency by the AXI interconnect infrastructure of the PYNQ system. This capped the maximum achievable clock frequency at **215 MHz**.

At this frequency, **Design 3** seems to be the optimal choice. It has the maximum achievable throughput of:

$$27.52 \text{ Gbps} = 128 \text{ bits} \times 215 \text{ MHz}$$

identical to that of Design 4, but with a **lower initial latency** (21 cycles vs. 31 cycles) and a more efficient area utilization. The additional internal speed of Design 4, while theoretically superior, provided no practical benefit due to the fixed system clock constraint, and introduced higher latency and resource overhead.

Design 3 and 4 were implemented in the PYNQ board using VITIS. The design was verified by sending a few plaintexts to the board using a serial input and the output was matched with the expected ciphertexts. These two designs are subitted for evaluation along with test bench and VITIS code.

7 Conclusion

The given baseline code was optimized and implemented in PYNQ board. The frequency was optimized from 10 MHz to 375 MHz. However, this process increased the area utilization, but due to high frequency, the throughput and initial latency of the design decreased significantly.

Disclaimer

Some portions of the code used in the assignment were generated with the help of AI tools such as Chat-GPT and Gemini. The AI tools were only used to generate some logic, most of the code is still the same as baseline code.